Mathematical Notes

Pablo, Arka

July 10, 2026

Abstract

First draft of the research papers to emerge from this project: Rao–Blackwellization, branching and poker case-study.

Contents

1 Introduction **2**

2 Game-Theoretical Model **2**

3 Self-Play Policy-Gradient Methods in Synchronous Rollouts **4**

3.1 Theoretical Model 4

3.2 Usual Implementation 6

3.3 Rao–Blackwellization of the Private State 8

3.4 Variance-Guided Branching 11

4 Poker: a Perfect Example **19**

1 Introduction

In the context of end-to-end policy learning, we train a model to learn a parametrized strategy profile π_θ. All policy-gradient methods to train such θ are based on a similar principle. They create a sequence $\theta_0, \dots \theta_k$ of parameters where typically θ_0 is a random seed, and they gradually build the next iterate by:

(i) **Rollout:** Sampling games using the current strategy profiles $(\pi_{\theta_i})_{i\leq k}$.

(ii) **Loss design:** Secretly, unbeknownst to the opponents, some new player with strategy π_θ will randomly replace the acting player. We pick some sort of loss function $\mathcal{L}_{k+1}(\theta)$ which depends on $(\theta_i)_{i\leq k}$, and which evaluates how well the new player π_θ is doing.

(iii) **Gradient Descent**: Using minibatches sampled from the online rollout database, we compute some estimator of the loss $\widehat{\mathcal{L}}_{k+1}(\theta)$ and descend along it.

Trajectories in games typically only get assigned to a computable reward at the end of the round (terminal trajectories), which forces rollout to *sample full runouts*. Typically, each runout is sampled by generating an explicit sequence of full game states (all private and public information is decided).

We introduce here two methods to reduce the variance of the loss estimators $\widehat{\mathcal{L}}_{k+1}(\theta)$:

(i) **Rao–Blackwellization of private state:** games have typically a very wide variance coming from the unavailable private information: the exact same strategy can lead to terrible failure or wonderful success depending on the private state of the opponents. We suggest addressing this by integrating away this variance. We replace private-state sampling in the loss estimator by conditional expectations over the posterior distribution of private states.

(ii) **Variance-guided Branching:** a significant amount of variance in the loss estimator typically also comes from the variety of public events (actions and chance events) which can significantly modify the strength of the acting player's private state. We suggest addressing this focusing the sampling around higher variance spots, by seeing rollouts as a tree instead of a set of sequences.

2 Game-Theoretical Model

To simplify the notations in this exposition, we focus on **finite Public-action Bayesian Games (PBG)**, although the same ideas can be applied to more general classes of games. In a couple words, this means: the game starts with a private chance event (the only source of imperfect-information) but then is played openly with all *players' action* and *chance* events being public. This class includes many poker, bidding, and auction models in which all private information is sampled initially and every subsequent action and chance outcome is public.

Definition 2.1 (Finite public-action Bayesian game)**.** For a finite set E, let

$$\Delta(E) := \left\{ \lambda : E \to [0,1] \;\middle|\; \sum_{e\in E} \lambda(e) = 1 \right\}. \tag{2.1}$$

A finite public-action Bayesian game consists of the following data.

(i) **Tree:** A finite prefix-closed set $\mathcal{S}$ of words, seen as a tree, with empty word $\varnothing \in \mathcal{S}$. Its elements are called public states, or trajectories. Write $\sigma \preccurlyeq \tau$ if σ is a prefix of τ,

and $\sigma \prec \tau$ if $\sigma \preccurlyeq \tau$ and $\sigma \neq \tau$. The terminal trajectories are the maximal elements of $\mathcal{S}$ and are denoted by $\mathcal{T}$. For every nonterminal trajectory $\sigma \in \mathcal{S} \setminus \mathcal{T}$, there is a finite event set $\mathcal{E}_\sigma$ such that the immediate successors of σ are precisely the trajectories σe, with $e \in \mathcal{E}_\sigma$.

(ii) **Owner:** A finite set $\mathcal{P}$ of *players* and an owner map $o : \mathcal{S} \setminus \mathcal{T} \to \mathcal{P} \sqcup \{\mathsf{c}\}$. If $o(\sigma) \in \mathcal{P}$ (resp. $\{\mathsf{c}\}$) is a player node (resp. chance node), we typically denote the elements of $\mathcal{E}_\sigma$ by a (resp. c). For $\sigma \in \mathcal{S}$ and $p \in \mathcal{P}$, we write $|\sigma|_p$ for the number of player nodes in σ owned by p, and $|\sigma|_{\text{player}}$ for the total number of player nodes in σ. For $\tau \in \mathcal{T}$, write $\tau^{\mathsf{pre-c}}$ for the largest prefix of τ which ends in no more than one chance or terminal node (for example $(caccaaaccct)^{\mathsf{pre-c}} = caccaaac$).

(iii) **Initial Private State Profile:** For every player $p \in \mathcal{P}$, a finite type set $\mathcal{X}_p$, and a distribution μ on $\mathcal{X} := \prod_{p=1}^{P} \mathcal{X}_p$. Elements of $\mathcal{X}$ are typically written $x = x_{\mathsf{c}} = (x_p)_{p \in \mathcal{P}}$, with $x_p \in \mathcal{X}_p$.

(iv) **Public-Chance Rules:** For every chance node σ and every initial profile $x \in \mathcal{X}$, a public-chance rule $\chi(\cdot \mid x, \sigma) \in \Delta(\mathcal{E}_\sigma)$.

(v) **Reward:** a function $\mathbf{R} = (R_p)_{p \in \mathcal{P}} : \mathcal{X} \times \mathcal{T} \to \mathbb{R}^{\mathcal{P}}$.

Definition 2.2 (Strategy and Random Trajectories)**.** A strategy profile is the data of $\pi(\cdot \mid x_p, \sigma) \in \Delta(\mathcal{E}_\sigma)$ for $\sigma \in o^{-1}(\mathcal{P})$ and $x_{o(\sigma)} \in \mathcal{X}_{o(\sigma)}$. For $p \in \mathcal{P}$, we write π_p for the strategy of player p, *i.e.* the subcollection of these distributions for $p = o(\sigma)$, and π_{-p} for the strategies of all other players. A strategy profile induces a joint probability measure $\mathbb{P}^\pi$ on the set $\mathcal{X} \times \mathcal{T}$.

For a trajectory $\sigma \in \mathcal{S}$, a player p and a private initial profile $x_p \in \mathcal{X}_p$, define the **reach** as the probability that player p acted according to σ conditional on initial profile x_p and every other event following σ:

$$r_p^\pi(\sigma \mid x) := \mathbb{P}^\pi(p \text{ acts as } \sigma \mid x_p, \text{other events follow } \sigma) = \prod_{\substack{\rho a \preccurlyeq \sigma \\ o(\rho) = p}} \pi(a \mid x_p, \rho), \tag{2.2}$$

where the product ranges over the actions of player p in the trajectory σ. Similarly, we define the **draw odds** of a trajectory σ as the probability that the public chance events in σ occurred conditional on the initial profile x:

$$\begin{aligned} r_{\mathsf{c}}(\sigma \mid x) &:= \mathbb{P}^\pi(\text{chance happens as in } \sigma \mid x, \text{players act as } \sigma) \\ &= \prod_{\substack{\rho c \preccurlyeq \sigma \\ o(\rho) = \mathsf{c}}} \chi(c \mid x, \rho), \end{aligned} \tag{2.3}$$

The expected payoff of player p is

$$J_p(\pi) := \mathbb{E}^\pi(R_p) = \mathbb{E}_{X \sim \mu}\left(\sum_{\tau \in \mathcal{T}} R_p(X, \tau) \prod_{q \in \mathcal{P} \cup \{c\}} r_q(\tau \mid X_q) \right) \tag{2.4}$$

For $\sigma \in \mathcal{S}$, $p = o(\sigma)$ and $x_p \in \mathcal{X}_p$, we write $V(x, \sigma) = \mathbb{E}^\pi(R_{o(\sigma)}(x, \tau) \mid x_p, \sigma)$. For a further choice of $a \in \mathcal{E}_\sigma$, we write $Q(x_p, \sigma a) = \mathbb{E}^\pi(R_{o(\sigma)}(x, \tau) \mid x_p, \sigma a)$ and $A(x_p, \sigma a) = Q(x_p, \sigma a) - V(x_p, \sigma)$.

Notation 2.3 (Ideal Random Variables)**.** We typically denote $X \sim \mu$ for a random initial profile. From it, we can define various other random variables successively. Firstly, we define $P^\pi \in \mathcal{P}$ by

$$\mathbb{P}(P^\pi = p \mid X) = \frac{\mathbb{E}^\pi(|\tau|_p \mid X)}{\mathbb{E}^\pi(|\tau|_{\text{player}} \mid X)} \tag{2.5}$$

Secondly, we denote Σ^π for the stationary random P^π-node of the Markov chain defined by the subtree $o^{-1}(P^\pi)$, conditioned at X, where the game is seen to constantly restart at terminal nodes. Thirdly, we let $A^\pi \sim \pi(\cdot \mid X_{P^\pi}, \Sigma^\pi)$. And finally, we let $\mathsf{T}^\pi \sim \frac{|\cdot|_{P^\pi}\mathbb{P}^\pi(\cdot|X)}{\mathbb{E}^\pi(|\tau|_{P^\pi}|X)}$ be a random terminal trajectory continuing $\Sigma^\pi A^\pi$ defined by

$$\mathbb{P}(\mathsf{T}^\pi = \tau \mid \Sigma^\pi A^\pi) = \mathbb{P}^\pi(\tau \mid \Sigma^\pi A^\pi) \tag{2.6}$$

It's easy to see that the law of Σ^π is given by

$$\mathbb{P}(\Sigma^\pi = \sigma \mid P^\pi) = \mathbb{1}_{o(\sigma)=P^\pi} \frac{\mathbb{P}^\pi(\sigma \preccurlyeq \tau)}{\mathbb{E}^\pi(|\tau|_{P^\pi})} \tag{2.7}$$

which is indeed a probability distribution as

$$\begin{aligned} \sum_{o(\sigma)=P^\pi} \mathbb{P}^\pi(\sigma \preccurlyeq \tau) &= \sum_{o(\sigma)=P^\pi} \sum_{\sigma \preccurlyeq \tau \in \mathcal{T}} \mathbb{P}^\pi(\tau) \\ &= \sum_{\tau \in \mathcal{T}} \mathbb{P}^\pi(\tau)|\tau|_{P^\pi} \\ &= \mathbb{E}^\pi(|\tau|_{P^\pi}) \end{aligned} \tag{2.8}$$

When π is clear from the context, we drop the superscript and write P, Σ, A and T. Note that T does not follow the law $\mathbb{P}^\pi$.

Recall below how the key theorem of policy gradient theory adapts to this setting.

Theorem 2.1 (Policy Gradient Theorem for Games)

For $p \in \mathcal{P}$, we have

$$\nabla_{\pi_p} J_p(\pi) = -\mathbb{E}^\pi(|\tau|_p)\nabla_{\pi_p}\mathbb{E}(\log(\pi(A \mid X_p, \Sigma))\,\mathrm{sg}[Q(X_p, \Sigma A)] \mid P = p) \tag{2.9}$$

3 Self-Play Policy-Gradient Methods in Synchronous Rollouts

3.1 Theoretical Model

Assuming a *synchronous rollout*, all main policy-gradient methods to solve games can be modelled by the following choices:

1. A **model** is the choice of a parametrized strategy profile $\theta \mapsto \pi^\theta$. Optionally, it also included the choice of a parametrized value functions $\theta \mapsto V^\theta$ meant to approximate $\mathbb{E}^{\pi^\theta}(R_p(x, \tau) \mid x_p, \sigma \preccurlyeq \tau)$ for all $\sigma \in \mathcal{S}$, $p = o(\sigma)$ and $x_p \in \mathcal{X}_p$.

2. Some **value** estimators

$$\widehat{A}(x_p, \sigma a) \simeq \mathbb{E}^\pi(R_p(x,\tau) \mid x_p, \sigma a) - \mathbb{E}^\pi(R_p(x,\tau) \mid x_p, \sigma) \tag{3.1}$$
$$\widehat{Q}(x_p, \sigma a) \simeq \mathbb{E}^\pi(R_p(x,\tau) \mid x_p, \sigma a) \tag{3.2}$$
$$\widehat{V}(x_p, \sigma) \simeq \mathbb{E}^\pi(R_p(x,\tau) \mid x_p, \sigma) \tag{3.3}$$

for $\sigma a \in \mathcal{S}$, $p = o(\sigma)$ and $x_p \in \mathcal{X}_p$. Optionally, the reward R_p above can be replaced by a modified reward depending on π and some reference profile ρ (see R-NaD).

3. A **loss function** $\mathcal{L}$ from which we want to estimate

$$l(\theta) = \mathbb{E}\Big(\mathcal{L}\Big((\pi^\theta, V^\theta, \pi, V)(X_P, \Sigma), A, \widehat{A}, \widehat{Q}, \widehat{V}\Big)\Big) \tag{3.4}$$

for Σ, A, P, X as in Notation 2.3 and V all based on a frozen policy π, called the *rollout profile* (typically an older version of π^θ), and for $\widehat{A} = \widehat{A}(X_P, \Sigma A)$, $\widehat{Q} = \widehat{Q}(X_P, \Sigma A)$ and $\widehat{V} = \widehat{V}(X_P, \Sigma)$. Optionally, the loss function may also depend on $\alpha(\cdot \mid X_P, \Sigma)$ for reference profile α (see NashPG).

The **training** is then an inductive process defining a sequence of weights $(\theta_k)_{k\geq 0}$. The first choice θ_0 is a random seed sampled from some initial distribution. Assuming we constructed $\theta_0, \dots, \theta_k$, we define the next iterate θ_{k+1} by: estimating $l(\theta)$ using the rollout profile θ_k, then performing some number of gradient descent steps on θ to minimize the estimated loss.

Assumptions. In the model above, we assume a few things that we justify here:

1. **No Q-head:** In many games of interests, the action space is too large to have a reasonnable Q-head as output of the model.

2. **No other-player value head:** With imperfect-information games, the state of the other player is not known. To output a value function for each player, one would need to $\sum_p |\mathcal{X}_p|$ outputs, which is typically too large to be practical. It's also not a quantity that we necessarily want the model to spend effort to learn: the expected returns over other players **without** conditioning on their private state is a much more relevant quantity to learn (see Equation 3.23).

3. **Synchronous rollout:** This is purely a theoretical assumption to simplify the exposition.

These assumptions shape the estimators and losses that are relevant for our context.

Primitives. For $0 \leq i \leq k$, we let $\rho_i^\theta = \frac{\pi^\theta(A|X_P,\Sigma)}{\pi^{\theta_i}(A|X_P,\Sigma)}$. We define **centered logits** as

$$z^\theta(a \mid X_P, \Sigma) := \log \pi^\theta(a \mid X_P, \Sigma) - \frac{1}{|\mathcal{E}_\Sigma|} \sum_{b \in \mathcal{E}_\Sigma} \log \pi^\theta(b \mid X_P, \Sigma) \tag{3.5}$$

Write $\widehat{A} = \widehat{Q}(X_P, \Sigma A) - V^{\theta_k}(X_P, \Sigma)$. If we are able to compute one, for $b \in \mathcal{E}_\Sigma$, write $\widehat{Q}(b) = \widehat{Q}(X_P, \Sigma b)$ an estimator of an all-action target. We also define a clipped value function $V^\theta_{\texttt{clip}} = V^{\theta_k} + \mathrm{clip}(V^\theta - V^{\theta_k}, -\varepsilon_V, \varepsilon_V)$. We then have the following classic losses (the notation

$\mathcal{L}[\widehat{E}]$ means "the loss $\mathcal{L}$ depends on a choice of estimator $\widehat{E}$"):

$$
\begin{aligned}
\mathcal{L}_{\texttt{PG}}[\widehat{A}] &:= -\rho_k^\theta \widehat{A}, \\
\mathcal{L}_{\texttt{clip}}[\widehat{A}] &:= -\min\Big\{\rho_k^\theta \widehat{A}, \operatorname{clip}\Big(\rho_k^\theta, 1-\varepsilon, 1+\varepsilon\Big)\widehat{A}\Big\} \\
\mathcal{L}_{\texttt{NeuRD}}[\widehat{Q}(-)] &:= -\sum_{b\in\mathcal{E}_\Sigma} z^\theta(b \mid X_P, \Sigma)\cdot \operatorname{clip}\Big(\widehat{Q}(b), -c_{\texttt{NeuRD}}, c_{\texttt{NeuRD}}\Big) \\
\mathcal{L}^{\texttt{sample}}_{\texttt{NeuRD}}[\widehat{Q}] &:= -\frac{z^\theta(A \mid X_P, \Sigma)}{\pi^{\theta_k}(A \mid X_P, \Sigma)}\cdot \operatorname{clip}\Big(\widehat{Q}, -c_{\texttt{NeuRD}}, c_{\texttt{NeuRD}}\Big) \\
\mathcal{L}_{\texttt{V}}[\widehat{V}] &:= c_V\Big(V^\theta(X_P,\Sigma) - \widehat{V}\Big)^2, \\
\mathcal{L}_{\texttt{V-clip}}[\widehat{V}] &:= c_V \max\Big\{\Big(V^\theta(X_P,\Sigma) - \widehat{V}\Big)^2, \Big(V^\theta_{\texttt{clip}}(X_P,\Sigma) - \widehat{V}\Big)^2\Big\}, \\
\mathcal{L}_{\texttt{H}} &:= -c_H\,\mathrm{H}\Big(\pi^\theta(\cdot \mid X_P,\Sigma)\Big), \\
\mathcal{L}_{\texttt{KL}} &:= c_{\texttt{KL}}\,\mathrm{KL}\Big(\pi^\theta(\cdot \mid X_P,\Sigma)\Big\|\pi^{\theta_{\mathrm{ref}(k)}}(\cdot \mid X_P,\Sigma)\Big) \\
\mathcal{L}^{\texttt{sample}}_{\texttt{KL}} &:= c_{\texttt{KL}}\Big(\rho_k^\theta \log(\rho_k^\theta) - \rho_k^\theta + 1\Big)
\end{aligned}
\tag{3.6}
$$

Relation to Theorem 2.1. The goal of this loss function is that its gradient resembles the gradient of the expected payoff $J_p(\pi_p^\theta, \mathrm{sg}[\pi_{-p}^\theta])$.

Practicality. In practice, it's essentially impossible to sample Σ in a non-biased way (one needs to first know the average length of the game). It's also impossible to create the estimators $\widehat{Q}$ and $\widehat{V}$ without seeing the sampled Σ as part of a full trajectory. These two constraints pushes us towards the approximations given in the following subsections.

3.2 Usual Implementation

We explain here the usual implementation of a training algorithm.

Algorithm 1: $\mathfrak{T} \leftarrow \texttt{SampleTerminal}(\pi, N)$

```
/* Terminal trajectory sampling used by ordinary sampled-private-state training.  */
```

Parameters: π, rollout profile; N, number of independent terminal rollouts

Output: $\mathfrak{T} = ((X_i, \mathsf{T}_i))_{i=1}^N$

for $i = 1, \ldots, N$ **do**
  Sample $X_i \sim \mu$
  Set $\sigma \leftarrow \emptyset$
  while $\sigma \notin \mathcal{T}$ **do**
    $p \leftarrow o(\sigma)$
    Sample

$$E \sim \begin{cases} \pi(\cdot \mid X_{i,p}, \sigma) & \text{if } o(\sigma) \in \mathcal{P} \\ \chi(\cdot \mid X_i, \sigma) & \text{otherwise} \end{cases}$$

    Set $\sigma \leftarrow \sigma E$
  end
  Set $\mathsf{T}_i \leftarrow \sigma$
end
return $\mathfrak{T}$

Definition 3.1 (Empirical terminal-rollout decision law)**.** Fix a terminal rollout batch $\mathfrak{T} = ((X_i, \mathsf{T}_i))_{i=1}^N$ produced by algorithm 1. Define an index $I \in \{1, \dots, N\}$ by

$$\mathbb{P}(I = i \mid \mathfrak{T}) = \frac{|\mathsf{T}_i|_{\text{player}}}{\sum_{j=1}^N |\mathsf{T}_j|_{\text{player}}}. \tag{3.7}$$

Then let $\Sigma A \preccurlyeq \mathsf{T}_I$ be a uniform choice of a player decision and $P = o(\Sigma)$.

Blank convergence statement: as $N \to \infty$, the law of Σ_N should converge to the ideal stationary-node law of Notation 2.3.

Primitives. For $\lambda \in [0,1]$, $1 \le i \le N$, $\sigma a \preccurlyeq \mathsf{T}_i$ and $p = o(\sigma)$, we write

$$\widehat{Q}^\lambda(X_{i,p}, \sigma a) := \lambda^{|\mathsf{T} \setminus \Sigma|_p} R_p(X_i, \mathsf{T}_i) + (1-\lambda) \sum_{\substack{\sigma \prec \rho \preccurlyeq \mathsf{T}_i \\ o(\rho) = p}} \lambda^{|\rho \setminus \sigma|_p - 1} V^{\theta_k}(X_{i,p}, \rho) \tag{3.8}$$

and $\widehat{A}^\lambda = \widehat{Q}^\lambda(X_{I,P}, \Sigma A) - V^{\theta_k}(X_{I,P}, \Sigma)$ and $\widehat{V}^\lambda = \widehat{Q}^\lambda(X_{I,P}, \Sigma A)$. Note that $\lambda = 1$ recovers the Monte Carlo return estimator (necessary in the absence of a value head). For $|\mathcal{P}| = 2$, $\eta > 0$ and $(x, \tau) \in \mathcal{X} \times \mathcal{T}$, we define the **R-NaD modified return** as

$$R_p^{\mathtt{NaD}}(x, \tau) := R_p(x, \tau) + \eta \sum_{\substack{\sigma a \preccurlyeq \mathsf{T} \\ o(\sigma) \in \mathcal{P}}} (1 - 2\mathbb{1}_{o(\sigma)=p}) \log\left(\frac{\pi^{\theta_k}(a \mid x_{o(\sigma)}, \sigma)}{\pi^{\theta_{\text{ref}(k)}}(a \mid x_{o(\sigma)}, \sigma)} \right) \tag{3.9}$$

Using this instead of R in the formula above, we get $\widehat{Q}^{\eta,\lambda}_{\mathtt{NaD}}$, $\widehat{A}^{\eta,\lambda}_{\mathtt{NaD}}$ and $\widehat{V}^{\eta,\lambda}_{\mathtt{NaD}}$. For $b \in \mathcal{E}_\Sigma$, we define the **R-NaD all-action return**

$$\begin{aligned} \widehat{Q}^{\eta,\lambda}_{\mathtt{NaD}}(b) = V^{\theta_k}(X_{I,P}, \Sigma) &- \eta \log\left(\frac{\pi^{\theta_k}(b \mid X_{I,P}, \Sigma)}{\pi^{\theta_{\text{ref}(k)}}(b \mid X_{I,P}, \Sigma)} \right) \\ &+ \frac{\mathbf{1}_{\{b=A\}}}{\pi^{\theta_k}(b \mid X_{I,P}, \Sigma)} \left(\widehat{A}^{\eta,\lambda}_{\mathtt{NaD}} + \eta \log\left(\frac{\pi^{\theta_k}(b \mid X_{I,P}, \Sigma)}{\pi^{\theta_{\text{ref}(k)}}(b \mid X_{I,P}, \Sigma)} \right) \right). \end{aligned} \tag{3.10}$$

Choices. Here are typical choices of losses used in training.

method	ref(k)	V^θ	minimized loss
`REINFORCE`	$\varnothing$	$\varnothing$ or fixed baseline	$\mathcal{L}_{\mathtt{PG}}[\widehat{A}^1]$
`Actor-critic`	$\varnothing$	V^θ	$\mathcal{L}_{\mathtt{PG}}[\widehat{A}^\lambda] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^\lambda] + \mathcal{L}_{\mathtt{H}}$
`PPO`	$\varnothing$	V^θ	$\mathcal{L}_{\mathtt{clip}}[\widehat{A}^\lambda] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^\lambda] + \mathcal{L}_{\mathtt{H}}$
`MMD-style PPO`	$\varnothing$	V^θ	$\mathcal{L}_{\mathtt{clip}}[\widehat{A}^\lambda] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^\lambda] + \mathcal{L}_{\mathtt{H}} + \mathcal{L}^{\mathtt{sample}}_{\mathtt{KL}}$
`NashPG-PPO`	$K\lfloor k/K \rfloor$	V^θ	$\mathcal{L}_{\mathtt{clip}}[\widehat{A}^\lambda] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^\lambda] + \mathcal{L}_{\mathtt{KL}}$
`RNaD-PPO`	$\text{ref}(k)$	V^θ	$\mathcal{L}_{\mathtt{clip}}[\widehat{A}^{\mathtt{NaD},\lambda}] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^{\mathtt{NaD},\lambda}]$
`R-NaD / DeepNash`	$\text{ref}(k)$	V^θ	$\mathcal{L}_{\mathtt{NeuRD}}[\widehat{Q}^{\eta,\lambda}_{\mathtt{NaD}}(-)] + \mathcal{L}_{\mathtt{V}}[\widehat{V}^{\mathtt{NaD},\lambda}]$

In `R-NaD`, it's not clear what function $\text{ref}(k)$ to choose as the original implementation is `IMPALA`-style as opposed to synchronous.

Implementation. Below is a pseudocode of the usual implementation.

Algorithm 2: $(\theta_k)_{k\geq 0} \leftarrow \texttt{SequentialPG}(\theta_0)$

```
/* Sequential rollout policy-gradient template corresponding to the procedure above.
   */
```

Parameters: θ_0, initial weights; θ, working copy of the next iterate
Hyperparameters: N, rollout count; B, minibatch size; chosen number of gradient-descent steps per iterate
Output: $(\theta_k)_{k\geq 0}$

for $k = 0, 1, 2, \ldots$ **do**
  Sample $(X_i, \mathsf{T}_i)_{i=1}^N \overset{\text{iid}}{\sim} (X, \mathsf{T}^{\pi^{\theta_k}})$ using $\texttt{SampleTerminal}(\pi^{\theta_k}, N)$
  for $i = 1, \ldots, N$ **do**
    Compute $R(X_i, \mathsf{T}_i)$ (optionally, using Equation 3.9)
    By backward propagation, for $\sigma a \preccurlyeq \mathsf{T}_i$ and $p = o(\sigma)$, compute $\widehat{Q}(X_{i,p}, \sigma a)$ using Equation 3.8
    Compute $\widehat{V}(X_{i,p}, \sigma)$
  end
  $\theta \leftarrow \theta_k$
  for *each gradient-descent step* **do**
    Sample $(I_b, \Sigma_b, A_b, P_b)_{b=1}^B \overset{\text{iid}}{\sim} (I, \Sigma, A, P)$ as in Definition 3.1
    Deduce $\widehat{Q}_b = \widehat{Q}(X_{I,P}, \Sigma A)$ and $\widehat{V}_b = \widehat{V}(X_{I,P}, \Sigma)$
    Estimate

$$\widehat{l}(\theta) = \frac{1}{B}\sum_{b=1}^{B} \mathcal{L}\Big((\pi^{\theta}, V^{\theta}, \pi^{\theta_k}, V^{\theta_k})(X_{I_b,P_b}, \Sigma_b), A_b, \widehat{Q}_b, \widehat{V}_b\Big)$$

    Descend θ along $\nabla_\theta \widehat{l}(\theta)$
  end
  store $\theta_{k+1} = \theta$
end
return $(\theta_k)_{k\geq 0}$

Optionally, for some $\mathrm{ref}(k) < k$ reference index, during minibatches, we can evaluate $\pi^{\theta_{\mathrm{ref}(k)}}(\cdot \mid X_{I,P}, \Sigma)$ at the sampled (I, P, Σ) to modify the loss function (see NashPG).

3.3 Rao–Blackwellization of the Private State

Motivation. Game strategies often involve disguising one's private information. In our leading example, poker, good players must "bluff" (pretend to have good cards when they don't), "slowplay" (pretend to have bad cards when they don't) and more generally, they must "protect their action ranges", meaning that $\mathbb{P}(\text{private state of } p \mid \text{public information})$ should not be too revealing. When learning poker, players are taught to "balance their ranges" and think "what would I do if I had a different hand?". Motivated by this, it's a natural choice to design models so that it doesn't see his private state x_p but rather decides what to do with any potential private state x_p. In other words, we do a currying transformation and see the model as

$$\theta \longmapsto \Big(x_p \in \mathcal{X}_p \mapsto \pi^{\theta}(\cdot \mid x_p, \sigma) \in \Delta(\mathcal{E}_\sigma)\Big) \tag{3.11}$$

which means that a single model run now outputs a collection of distributions. The goal of Rao–Blackwellization is to exploit this architecture in the training algorithm to reduce the variance of the loss estimator induced by the private state X.

Algorithm 3: $\mathfrak{T}^{\mathsf{pre-c}} \leftarrow \texttt{SamplePreTerminal}(\pi, N)$

```
/* Public pre-terminal sampling for Rao-Blackwellized training.  */
```

Parameters: π, rollout profile; N, number of independent public rollouts

Output: $\mathfrak{T}^{\mathsf{pre-c}} = ((\mathsf{T}_i^{\mathsf{pre-c}}, \mathfrak{r}_i))_{i=1}^{N}$

for $i = 1, \dots, N$ **do**
  Sample $X_i \sim \mu$
  Set $\sigma \leftarrow \emptyset$ and $\mathfrak{r}_i \leftarrow 1$
  while $\sigma \notin \mathcal{T}^{\mathsf{pre-c}}$ **do**
    Sample

$$E \sim \begin{cases} \pi(\cdot \mid X_{i,p}, \sigma) & \text{if } o(\sigma) \in \mathcal{P} \\ \chi(\cdot \mid X_i, \sigma) & \text{otherwise} \end{cases}$$

    Set $\sigma \leftarrow \sigma E$ and

$$\mathfrak{r}_{i,o(\sigma)} \leftarrow \mathfrak{r}_{i,o(\sigma)} \odot \begin{cases} \pi(E \mid -, \sigma) & \text{if } o(\sigma) \in \mathcal{P} \\ \chi(E \mid -, \sigma) & \text{otherwise} \end{cases}$$

    Normalize $\mathfrak{r}_{i,o(\sigma)}$
  end
  Set $\mathsf{T}_i^{\mathsf{pre-c}} \leftarrow \sigma$
end
return $\mathfrak{T}^{\mathsf{pre-c}}$

Definition 3.2 (Empirical pre-terminal Rao–Blackwell decision law)**.** Fix a pre-terminal rollout batch $\mathfrak{T}^{\mathsf{pre-c}} = ((\mathsf{T}_i^{\mathsf{pre-c}}, \mathfrak{r}_i))_{i=1}^{N}$ produced by algorithm 3. Define $I \in \{1, \dots, N\}$ by

$$\mathbb{P}(I = i \mid \mathfrak{T}^{\mathsf{pre-c}}) = \frac{|\mathsf{T}_i^{\mathsf{pre-c}}|_{\text{player}}}{\sum_{j=1}^{N} |\mathsf{T}_j^{\mathsf{pre-c}}|_{\text{player}}}. \tag{3.12}$$

Then let $\Sigma A \preccurlyeq \mathsf{T}_I^{\mathsf{pre-c}}$ be a uniform choice of a player decision and $P = o(\Sigma)$.

Blank convergence statement: as $N \to \infty$, the law of Σ should converge to the ideal stationary-node law of Notation 2.3, with terminal continuations replaced by their pre-terminal public prefixes.

Theory in the Ideal Setup. Essentially, we modify the expected value computations as

$$\begin{aligned} \widehat{Q} &= \mathbb{E}^{\pi}(R_p(x, \tau) \mid x_p, \sigma a) \\ &= \mathbb{E}^{\pi}(\mathbb{E}^{\pi}(R_p(x, \tau) \mid x_p, \tau^{\mathsf{pre-c}}) \mid \sigma a) \end{aligned} \tag{3.13}$$

$$\mathbb{E}(\mathcal{L}) = \mathbb{E}(\mathbb{E}^{\pi}(\mathcal{L}(x, \Sigma A) \mid \Sigma A)) \tag{3.14}$$

which reduces variance by the Law of Total Variance.

Primitives. For $\lambda \in [0,1]$, $1 \le i \le N$, $\sigma a \preccurlyeq \mathsf{T}_i$, $p = o(\sigma)$ and x_p compatible with T_i, we write

$$\widehat{Q}^{\lambda}(x_p, \sigma a) := \lambda^{|\mathsf{T}\setminus\Sigma|_p}\mathbb{E}^{\pi^{\theta_k}}(R_p(x,\tau) \mid x_p, \mathsf{T}_i^{\mathsf{pre-c}} \preccurlyeq \tau) + (1-\lambda)\sum_{\substack{\sigma \prec \rho \preccurlyeq \mathsf{T}_i \\ o(\rho)=p}} \lambda^{|\rho\setminus\sigma|_p - 1} V^{\theta_k}(x_p, \rho) \quad (3.15)$$

and $\widehat{A}^{\lambda}(x_p) = \widehat{Q}^{\lambda}_f(x_p, \Sigma A) - V^{\theta_k}(x_p, \Sigma)$. and $\widehat{V}^{\lambda}(x_p) = \widehat{Q}^{\lambda}(x_p, \Sigma A)$.

Monte Carlo Methods. After experimentation, we found that a sequential importance sampler (SIS) resulted in the fastest convergence of the Monte Carlo estimates. Other methods such as rejection sampling or MCMC were found to be less efficient.

Algorithm 4: $(\widehat{\mathbb{P}^{\mathfrak{r}}}, \widehat{V}) \leftarrow$ `PreTerminalMC`$(\mathfrak{r})$

`/* Black-box Monte Carlo completion from public pre-terminal prefixes. */`

Parameters: $\mathfrak{r}$, reach cache
Hyperparameters: Monte Carlo budget
Output: $(\widehat{\mathbb{P}^{\mathfrak{r}}}, \widehat{V})$

1 Use the a Monte Carlo simulation to sample $(X, \mathsf{T}) \sim \mathbb{P}^{\mathfrak{r}}$ with

$$\mathbb{P}^{\mathfrak{r}}(x,\tau) = \frac{\prod_{q\in[P]\cup\{c\}} r_q(\tau \mid x_q)}{\sum_y \prod_{q\in[P]\cup\{c\}} r_q(\tau \mid y_q)}$$

2 Store statistical occurance $\left(\widehat{\mathbb{P}^{\mathfrak{r}}}(x_p)\right)_{\substack{p\in\mathcal{P} \\ x_p\in\mathcal{X}_p}}$ and statistical average

$$\left(\widehat{V}(x_p) \simeq \mathbb{E}(R_p(X,\mathsf{T}) \mid X_p = x_p)\right)_{\substack{p\in\mathcal{P} \\ x_p\in\mathcal{X}_p}}$$

return $(\widehat{\mathbb{P}^{\mathfrak{r}}}, \widehat{V})$

Algorithm 5: $(\theta_k)_{k\geq 0} \leftarrow$ `SequentialRaoBlackwellPG`(θ_0)

```
/* Sequential rollout policy-gradient template corresponding to the procedure above.
   */
```

Parameters: θ_0, initial weights; θ, working copy of the next iterate
Hyperparameters: N, rollout count; B, minibatch size; chosen number of gradient-descent steps per iterate
Output: $(\theta_k)_{k\geq 0}$

for $k = 0, 1, 2, \ldots$ **do**
  Sample $(\mathsf{T}_i^{\mathsf{pre-c}}, \mathfrak{r}_i)_{i=1}^N \overset{\mathsf{iid}}{\sim} \mathsf{T}^{\pi^{\theta_k}}$ using `SamplePreTerminal`(π^{θ_k}, N)
  For $1 \leq i \leq N$, $p \in \mathcal{P}$ and $x_p \in \mathcal{X}_p$, estimate

$$\begin{aligned} \widehat{\mathbb{P}}(x_p \mid \mathsf{T}_i^{\mathsf{pre-c}}) &\simeq \mathbb{P}^{\pi}(x_p, \tau \mid \mathsf{T}_i^{\mathsf{pre-c}}) \\ \widehat{V}_p(x_p, \mathsf{T}_i^{\mathsf{pre-c}}) &\simeq \mathbb{E}^{\pi}(R_p(x,\tau) \mid x_p, \mathsf{T}_i^{\mathsf{pre-c}}) \end{aligned} \tag{3.16}$$

    using `PreTerminalMC`$(\mathfrak{r}_i)$
  for $i = 1, \ldots, N$ **do**
    By backward propagation, for $\sigma a \preccurlyeq \mathsf{T}_i$, $p = o(\sigma)$ and $x_p \in \mathcal{X}_p$, compute $\widehat{Q}(x_p, \sigma a)$ using Equation 3.15
    Compute $\widehat{V}(x_p, \sigma)$
  end
  $\theta \leftarrow \theta_k$
  for *each gradient-descent step* **do**
    Sample $(J_b, \Sigma_b, A_b, P_b)_{b=1}^B \overset{\mathsf{iid}}{\sim} (J, \Sigma, A, P)$ as in Definition 3.2
    Deduce $\widehat{Q}_b(-) = \widehat{Q}(-, \Sigma A)$ and $\widehat{V}_b(-) = \widehat{V}(-, \Sigma)$
    Estimate

$$\widehat{l}(\theta) = \frac{1}{B} \sum_{b=1}^{B} \sum_{x_{P_b}} \mathcal{L}\Big((\pi^{\theta}, V^{\theta}, \pi^{\theta_k}, V^{\theta_k})(x_{P_b}, \Sigma_b), A_b, \widehat{Q}_b, \widehat{V}_b\Big) \cdot \widehat{\mathbb{P}}(x_{P_b} \mid \mathsf{T}_{J_b}^{\mathsf{pre-c}})$$

    Descend θ along $\nabla_\theta \widehat{l}(\theta)$
  end
  store $\theta_{k+1} = \theta$
end
return $(\theta_k)_{k\geq 0}$

3.4 Variance-Guided Branching

Rao–Blackwellization integrates potential private states knowing a public terminal trajectory. It leaves on the table another substantial source of variance: a state σa might end in many different terminal trajectories. We suggest here to modify sequential rollout sampling into a Monte Carlo Tree Search (MCTS) style of sampling. This approach is particularly useful in games because typically there is only one possible initial public state, or a small space thereof (we build this into the PBG formalism where different initial public states are different games). For example, in our leading example of Texas Hold'em cash poker, the initial public state only contains the stack size of each player (and maybe antes, straddles or other house rules): this is a much smaller space than the space of all potential prompts for a coding model.

We further improve MCTS by suggesting a variance-guided branching strategy: we branch more on the nodes with high downstream reward variance.

Branched Rollouts. We enrich the rollout into a branching random walk on the public tree. The design constraint, encouraged by the Rao–Blackwellized pipeline, is that the enrichment must be *public*: any stored importance weight depending on a sampled hidden assignment would contaminate every posterior downstream (Warning 3.2).

Algorithm 6: $(\ell^{\beta,\pi} : \mathfrak{S}^{\beta,\pi} \to \mathcal{S}, N) \leftarrow \texttt{SampleTree}(\pi, \beta)$

```
/* Sampling procedure for the rollout tree of definition 3.3.                    */
```

Parameters: π, rollout profile; $\beta : \mathcal{S} \setminus \mathcal{T} \to [1, \infty)$, intensity

Output: sampled tree $\mathfrak{S}^{\beta,\pi}$, label map $\ell^{\beta,\pi} : \mathfrak{S}^{\beta,\pi} \to \mathcal{S}$, and child counts N

Set $\mathfrak{S}^{\beta,\pi} \leftarrow \{\emptyset\}$, $\ell^{\beta,\pi}(\emptyset) \leftarrow \emptyset$, and $\mathcal{F} \leftarrow \{\emptyset\}$

while $\mathcal{F} \neq \emptyset$ **do**

  Choose and remove some node $\mathsf{s} \in \mathcal{F}$ and set $\sigma \leftarrow \ell^{\beta,\pi}(\mathsf{s})$

  if σ *is pre-terminal* **then**

    Set $\mathfrak{E}_{\mathsf{s}} \leftarrow \emptyset$

  else

    Let $N^{\beta,\pi}(\mathsf{s}) \sim \lfloor \beta(\sigma) \rfloor + \text{Bernouilli}\left(\beta(\sigma) - \lfloor \beta(\sigma) \rfloor\right)$

    Set $\mathfrak{E}_{\mathsf{s}} \leftarrow \{\mathsf{a}_1, \ldots, \mathsf{a}_{N^{\beta,\pi}(\mathsf{s})}\}$ to be the set of children of s

    Let

$$(E_{\mathsf{a}})_{\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}} \overset{\text{iid}}{\sim} \begin{cases} \mathbb{E}^{\pi}(\pi(\cdot \mid x_p, \sigma) \mid \sigma) & \text{if } o(\sigma) \in \mathcal{P} \\ \mathbb{E}^{\pi}(\chi(\cdot \mid x, \sigma) \mid \sigma) & \text{otherwise} \end{cases}$$

    For $\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}$, let $\ell^{\beta,\pi}(\mathsf{sa}) = \sigma E_{\mathsf{a}}$

    $\mathcal{F} \leftarrow \mathcal{F} \sqcup \{\mathsf{sa} : \mathsf{a} \in \mathfrak{E}_{\mathsf{s}}\}$

  end

end

return $(\mathfrak{S}^{\beta,\pi}, \ell^{\beta,\pi} : \mathfrak{S}^{\beta,\pi} \to \mathcal{S}, N^{\beta,\pi} : \mathfrak{S}^{\beta,\pi} \to \mathbb{N})$

Remark 3.1 (Practical Implementation). Note that practically speaking, to sample each outgoing event E_{a}, we first sample a temporary private state x conditioned on σ (for example, in poker, we do this by rejection sampling), then we sample according to the chance rules and player strategies. The deterministic "1+" is a *spine*: it guarantees survival of every lineage, and at $\beta \equiv 1$ the tree is the set of nodes in a random trajectory $\mathsf{T} \sim \mathbb{P}^{\pi}(\cdot)$.

Definition 3.3 (Rollout tree)**.** Let $\beta : \mathcal{S} \setminus \mathcal{T} \to [1, \infty)$ be an intensity function and $(\mathfrak{S}^{\beta,\pi}, \ell^{\beta,\pi} : \mathfrak{S}^{\beta,\pi} \to \mathcal{S}, N^{\beta,\pi} : \mathfrak{S}^{\beta,\pi} \to \mathbb{N}) \leftarrow \texttt{SampleTree}(\pi, \beta)$ be a random labeled tree, seen as a Markov chain (with uniform transition probabilities). When β and π are clear, write ℓ for $\ell^{\beta,\pi}$. Conditioning on $\mathfrak{S}^{\beta,\pi}$ below means conditioning on the sampled labeled tree, including ℓ and $N^{\beta,\pi}$. We have a natural probability distribution

$$\mathbb{P}^{\beta,\pi}(x, \tau \mid \mathfrak{S}^{\beta,\pi}) = \mathbb{P}^{\beta,\pi}(\tau \mid \mathfrak{S}^{\beta,\pi}) \mathbb{P}^{\pi}(x \mid \tau)$$

on $\mathcal{X} \times \mathcal{T}$ induced from this Markov chain and given by

$$\mathbb{P}^{\beta,\pi}(\tau \mid \mathfrak{S}^{\beta,\pi}) = \sum_{\ell(\mathsf{t}) = \tau} \prod_{\mathsf{s} \prec \mathsf{t}} \frac{1}{N^{\beta,\pi}(\mathsf{s})} \tag{3.17}$$

Equivalently, let $\mathsf{T}^{\beta,\pi}$ be the sampled terminal node of this Markov chain and set $\mathsf{T}^{\beta,\pi} :=$

$\ell(\mathsf{T}^{\beta,\pi})$. From it, we can firstly define $P^{\beta,\pi} \in \mathcal{P}$ by

$$\mathbb{P}(P^{\beta,\pi} = p \mid \mathfrak{S}^{\beta,\pi}) = \frac{\mathbb{E}^{\beta,\pi}(|\tau|_p \mid \mathfrak{S}^{\beta,\pi})}{\mathbb{E}^{\beta,\pi}(|\tau|_{\text{player}} \mid \mathfrak{S}^{\beta,\pi})} \tag{3.18}$$

Secondly, we denote $\mathsf{S}^{\beta,\pi}$ for a random node sampled from the stationary distribution of the Markov chain defined on the sub-tree of $\mathfrak{S}^{\beta,\pi}$ on these nodes labeled by a game node owned by $P^{\beta,\pi}$, and we let $\Sigma^{\beta,\pi} = \ell(\mathsf{S}^{\beta,\pi})$ be its label. Thirdly, we let $\mathsf{A}^{\beta,\pi}$ be a uniform choice in $\mathfrak{E}_{\mathsf{S}^{\beta,\pi}}$ and let $A^{\beta,\pi}$ be its public event label, so $\ell(\mathsf{S}^{\beta,\pi}\mathsf{A}^{\beta,\pi}) = \Sigma^{\beta,\pi}A^{\beta,\pi}$. And finally, we let

$$\mathsf{T}^{\beta,\pi,\mathsf{pre-c}} \sim \frac{|\cdot|_{P^{\beta,\pi}}\mathbb{P}^{\beta,\pi}(\cdot \preccurlyeq \tau \mid \mathfrak{S}^{\beta,\pi})}{\mathbb{E}^{\beta,\pi}(|\tau|_{P^{\beta,\pi}} \mid \mathfrak{S}^{\beta,\pi})}$$

be a random pre-terminal trajectory continuing $\Sigma^{\beta,\pi}A^{\beta,\pi}$ defined by

$$\mathbb{P}(\mathsf{T}^{\beta,\pi} = \tau \mid \Sigma^{\beta,\pi}A^{\beta,\pi}, \mathfrak{S}^{\beta,\pi}) = \mathbb{P}^{\beta,\pi}(\tau \mid \Sigma^{\beta,\pi}A^{\beta,\pi}, \mathfrak{S}^{\beta,\pi}) \tag{3.19}$$

and $X \sim \mathbb{P}^{\pi}(\cdot \mid \mathsf{T}^{\beta,\pi,\mathsf{pre-c}})$ be a random initial profile. It's easy to see that the law of $\mathsf{S}^{\beta,\pi}$ is given by

$$\mathbb{P}(\mathsf{S}^{\beta,\pi} = \mathsf{s} \mid P^{\beta,\pi}, \mathfrak{S}^{\beta,\pi}) = \frac{\mathbb{1}_{o(\ell(\mathsf{s}))=P^{\beta,\pi}}}{\mathbb{E}^{\beta,\pi}(|\tau|_{P^{\beta,\pi}} \mid \mathfrak{S}^{\beta,\pi}) \prod_{\mathsf{r}\prec\mathsf{s}} N^{\beta,\pi}(\mathsf{r})} \tag{3.20}$$

which is indeed a probability distribution. When β and π are clear from context, we write $\mathcal{B}$ instead of $\mathcal{B}^{\beta,\pi}$.

Lemma 3.1 (Terminal node distribution)

We have

$$\mathbb{E}(\mathbb{P}^{\beta,\pi}(\cdot \mid \mathfrak{S}^{\beta,\pi})) = \mathbb{P}^{\pi}(\cdot) \tag{3.21}$$

Lemma 3.2 (Branch independence)

Fix a sampled-tree node s *with public label* σ *and sampled outgoing-action set* $\mathfrak{E}_{\mathsf{s}}$*, and put* $m = N(\mathsf{s}) = |\mathfrak{E}_{\mathsf{s}}|$*. Conditional on* σ *and* m*, the rooted child branches*

$$\left(\ell(\mathsf{sa}), \mathfrak{S}^{\beta,\pi}_{\mathsf{sa}\preccurlyeq}\right)_{\mathsf{a}\in\mathfrak{E}_{\mathsf{s}}}$$

are mutually independent; in particular, any statistic of one child branch is independent of any statistic of the sibling forest, conditional on the public parent and the chosen child label.

Warning 3.2 (Don't sample a true hand!). One might consider sampling X first and then generating the sub-tree using the knowledge that X is the true private state. While this still satisfies Lemma 3.1 marginally, it violates Lemma 3.2. The sub-trees sampled by this process would be "self-correlated": for $\mathsf{a} \neq \mathsf{b} \in \mathfrak{E}_{\mathsf{s}}$, the sub-trees $\mathfrak{S}_{\mathsf{sa}\preccurlyeq}$ and $\mathfrak{S}_{\mathsf{sb}\preccurlyeq}$ share the same sampled private state and would not be independent. In particular, the sibling average used as the leave-one-out baseline for $\widehat{A}^{\lambda}$ would no longer be independent of the target branch after conditioning on the target action label. While this allows us to compute Monte Carlo estimators for expected values (such as reward functions), it has much weaker guarantees of convergence due to the lack of locality.

Variance-Based Intensities (naïve version). Using Neyman allocation principles, we target nodes with larger continuation variance.

$$\beta(\sigma) \propto \sqrt{\operatorname{tr} \operatorname{Cov}^{\pi}(\mathbf{R}(x,\tau) \mid \sigma)}. \tag{3.22}$$

For player nodes, in order to estimate this variance, we two new heads:

$$\theta \longmapsto \mathbf{V}^{\theta}(\sigma) \simeq \mathbb{E}^{\pi}(\mathbf{R}(x,\tau) \mid \sigma) \tag{3.23}$$

and

$$\theta \longmapsto U^{\theta}(\sigma) \simeq \mathbb{E}^{\pi}(\|\mathbf{R}(x,\tau)\|_2^2 \mid \sigma). \tag{3.24}$$

We then use the identity

$$\operatorname{tr} \operatorname{Cov}^{\pi}(\mathbf{R}(x,\tau) \mid \sigma) = \mathbb{E}^{\pi}(\|\mathbf{R}(x,\tau)\|_2^2 \mid \sigma) - \|\mathbb{E}^{\pi}(\mathbf{R}(x,\tau) \mid \sigma)\|_2^2.$$

and define

$$s_k(\sigma) := \sqrt{\left[U^{\theta_k}(\sigma) - \|\bar{\mathbf{V}}^{\theta_k}(\sigma)\|_2^2\right]_+}$$

Practically speaking, we can't immidiately use these heads in training before they are somewhat trained. So we define on player nodes a function $b(\sigma) \in \mathcal{B}$ partitioning them into buckets. Then we define for $b \in \mathcal{B}$, some initial target branching intensity $\beta_{\mathtt{fix}}(b)$, and some maximum branching intensity $\beta_{\max}(b)$. During a warmup period, we use $\beta(\sigma) = \beta_{\mathtt{fix}}(b(\sigma))$. After this warmup period, for each bucket $b \in \mathcal{B}$, we define

$$\bar{s}_{k-1}(b) := \frac{\sum_{\rho \in \mathbf{S}_{k-1} : b(\rho) = b} s_{k-1}(\rho)}{|\{\rho \in \mathbf{S}_{k-1} : b(\rho) = b\}|}.$$

and we use

$$\beta_k(\sigma) := \operatorname{clip}\left(\beta_{\mathtt{avg}}(b(\sigma)) \frac{s_k(\sigma)}{\bar{s}_{k-1}(b(\sigma))}, 1, \beta_{\max}\right)$$

If the denominator is zero, we default to $\beta_{\mathtt{fix}}(\sigma)$. At chance nodes, keep fixed branching intensities to avoid introducing a new model.

Primitive Functions. Back to a synchronous rollout training, we fix weights $\theta_0, \dots, \theta_k$. We let $\mathfrak{S}$, ℓ, P, S, Σ, A, A, T, $\mathsf{T}^{\mathsf{pre-c}}$ and X be as above for $\pi = \pi^{\theta_k}$. For a sampled-tree node s, write $\mathfrak{E}_{\mathsf{s}}$ for its sampled outgoing-action set, so that the children are sa for $\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}$. For a player node $\mathsf{s} \in \mathfrak{S}$ with $\ell(\mathsf{s}) = \sigma$, a sampled action $\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}$ with $\ell(\mathsf{sa}) = \sigma a_{\mathsf{a}}$, $p = o(\sigma)$ and $x_p \in \mathcal{X}_p$, we

define the estimators

$$\begin{aligned}
\widehat{Q}^{\lambda}(x_p, \mathsf{sa}) &= \mathbb{E}^{\beta,\pi}\Big(\lambda^{|\tau\setminus\sigma|_p} R_p(x,\tau)\Big| x_p, \mathsf{sa} \preccurlyeq \mathsf{T}\Big) \\
&\quad + (1-\lambda)\mathbb{E}^{\beta,\pi}(|\tau\setminus\sigma|_p \mid \mathsf{sa} \preccurlyeq \mathsf{T})\mathbb{E}\Big(\lambda^{|\Sigma\setminus\sigma|_p - 1} V^{\theta_k}(X_p,\Sigma)\Big| \mathsf{sa} \preccurlyeq \mathsf{S}, P = p, \mathfrak{S}\Big) \\
\widehat{V}^{\lambda}(x_p, \mathsf{s}) &= \mathbb{E}^{\beta,\pi}\Big(\lambda^{|\tau\setminus\sigma|_p} R_p(x,\tau)\Big| x_p, \mathsf{s} \preccurlyeq \mathsf{T}\Big) \\
&\quad + (1-\lambda)\mathbb{E}^{\beta,\pi}(|\tau\setminus\sigma|_p \mid \mathsf{s} \preccurlyeq \mathsf{T})\mathbb{E}\Big(\lambda^{|\Sigma\setminus\sigma|_p - 1} V^{\theta_k}(X_p,\Sigma)\Big| \mathsf{s} \prec \mathsf{S}, P = p, \mathfrak{S}\Big) \\
\widehat{V}^{\lambda,\neg\mathsf{a}}(x_p, \mathsf{s}) &= \lambda\Bigg[\mathbb{E}^{\beta,\pi}\Big(\lambda^{|\tau\setminus\sigma|_p} R_p(x,\tau)\Big| x_p, \mathsf{s} \preccurlyeq \mathsf{T}, \mathsf{sa} \not\preccurlyeq \mathsf{T}\Big) \\
&\quad + (1-\lambda)\mathbb{E}^{\beta,\pi}(|\tau\setminus\sigma|_p | \mathsf{s} \preccurlyeq \mathsf{T}, \mathsf{sa} \not\preccurlyeq \mathsf{T})\mathbb{E}\Big(\lambda^{|\Sigma\setminus\sigma|_p - 1} V^{\theta_k}(X_p,\Sigma)\Big| \mathsf{s} \preccurlyeq \mathsf{S}, \mathsf{sa} \not\preccurlyeq \mathsf{S}, P = p, \mathfrak{S}\Big)\Bigg] + (1 \\
\widehat{V}^{\lambda,\neg\mathsf{a}}(x_p, \mathsf{s}) &= V^{\theta_k}(x_p, \sigma) \quad \text{if } N(\mathsf{s}) = 1 \\
\widehat{A}^{\lambda}(x_p, \mathsf{sa}) &= \widehat{Q}^{\lambda}(x_p, \mathsf{sa}) - \widehat{V}^{\lambda,\neg\mathsf{a}}(x_p, \mathsf{s}) \\
\widehat{\mathbf{V}}^{\lambda}(\mathsf{s}) &= \mathbb{E}^{\beta,\pi}\Big(\lambda^{|\tau\setminus\sigma|_{\text{player}}} \mathbf{R}(x,\tau)\Big| \mathsf{s} \preccurlyeq \mathsf{T}\Big) \\
&\quad + (1-\lambda)\mathbb{E}^{\beta,\pi}(|\tau\setminus\sigma|_{\text{player}} \mid \mathsf{s} \preccurlyeq \mathsf{T})\mathbb{E}\Big(\lambda^{|\Sigma\setminus\sigma|_{\text{player}}} \mathbf{V}^{\theta_k}(\Sigma)\Big| \mathsf{s} \prec \mathsf{S}, \mathfrak{S}\Big) \\
\widehat{U}^{\lambda}(\mathsf{s}) &= \mathbb{E}^{\beta,\pi}\Big(\lambda^{|\tau\setminus\sigma|_{\text{player}} - 1} \|\mathbf{R}(x,\tau)\|_2^2\Big| \mathsf{s} \preccurlyeq \mathsf{T}\Big) \\
&\quad + (1-\lambda)\mathbb{E}^{\beta,\pi}(|\tau\setminus\sigma|_{\text{player}} \mid \mathsf{s} \preccurlyeq \mathsf{T})\mathbb{E}\Big(\lambda^{|\Sigma\setminus\sigma|_{\text{player}} - 1} U^{\theta_k}(\Sigma)\Big| \mathsf{s} \prec \mathsf{S}, \mathfrak{S}\Big)
\end{aligned}$$

The $\widehat{Q}^{\lambda}$, $\widehat{V}^{\lambda}$, $\widehat{\mathbf{V}}^{\lambda}$ and $\widehat{U}^{\lambda}$ targets are unbiased by Lemma 3.1 (assuming that the V and U heads are perfect). For $N(\mathsf{s}) > 1$, Lemma 3.2 makes the sibling average entering $\widehat{V}^{\lambda,\neg\mathsf{a}}$ independent of the target branch after conditioning on the target action label, so $\widehat{A}^{\lambda}$ is an unbiased leave-one-branch advantage estimator. All of these targets can be computed by backward propagation (see algorithm below).

Now say we have a current parameter θ. For a sampled (P, S, Σ), a sampled action $\mathsf{a} \in \mathfrak{E}_{\mathsf{S}}$ labeled by Σa_{a}, and a private state $x_P \in \mathcal{X}_P$, we use the local aliases $\widehat{Q}(x_P, \mathsf{a}) := \widehat{Q}^{\lambda}(x_P, \mathsf{Sa})$, $\widehat{A}(x_P, \mathsf{a}) := \widehat{A}^{\lambda}(x_P, \mathsf{Sa})$, $\widehat{V}(x_P) := \widehat{V}^{\lambda}(x_P, \mathsf{S})$, $\widehat{\mathbf{V}} := \widehat{\mathbf{V}}^{\lambda}(\mathsf{S})$ and $\widehat{U} := \widehat{U}^{\lambda}(\mathsf{S})$. We also define new loss function

$$\begin{aligned}
\mathcal{L}_{\mathtt{U}}[\widehat{U}^{\lambda}] &= c_U\Big(U^{\theta}(\Sigma) - \widehat{U}^{\lambda}\Big)^2, \\
\mathcal{L}_{\mathbf{V}}[\widehat{\mathbf{V}}^{\lambda}] &= c_{\mathbf{V}}\|\mathbf{V}^{\theta}(\Sigma) - \widehat{\mathbf{V}}^{\lambda}\|_2^2.
\end{aligned} \tag{3.26}$$

as well as $\mathcal{L}_{\mathtt{U-clip}}$ and $\mathcal{L}_{\mathbf{V}-\mathtt{clip}}$, defined analogously to $\mathcal{L}_{\mathbf{V}-\mathtt{clip}}$.

Algorithm 7: $(\widehat{\mathbb{P}}, \widehat{V}, \widehat{\mathbf{V}}, \widehat{U}) \leftarrow$ `PreTerminalMomentMC`$(\mathfrak{r})$

`/* Black-box Monte Carlo completion from public pre-terminal prefixes, including the squared-norm moment. */`

Parameters: $\mathfrak{r}$, reach cache
Hyperparameters: Monte Carlo budget
Output: $(\widehat{\mathbb{P}^{\mathfrak{r}}}, \widehat{V}, \widehat{\mathbf{V}}, \widehat{U})$

1 Use a Monte Carlo simulation to sample $(X, \mathsf{T}) \sim \mathbb{P}^{\mathfrak{r}}$ with

$$\mathbb{P}^{\mathfrak{r}}(x, \tau) = \frac{\prod_{q \in [P] \cup \{c\}} r_q(\tau \mid x_q)}{\sum_y \prod_{q \in [P] \cup \{c\}} r_q(\tau \mid y_q)}$$

2 Store statistical occurrence $\left(\widehat{\mathbb{P}^{\mathfrak{r}}}(x_p)\right)_{\substack{p \in \mathcal{P} \\ x_p \in \mathcal{X}_p}}$, statistical average

$$\left(\widehat{V}(x_p) \simeq \mathbb{E}(R_p(X, \mathsf{T}) \mid X_p = x_p)\right)_{\substack{p \in \mathcal{P} \\ x_p \in \mathcal{X}_p}}$$

and statistical average

$$\widehat{U} \simeq \mathbb{E}(\|\mathbf{R}(X, \mathsf{T})\|_2^2)$$

3 Compute $\widehat{\mathbf{V}} \leftarrow \left(\widehat{\mathbb{P}^{\mathfrak{r}}}(-_p) \cdot \widehat{V}(-_p)\right)_{p \in \mathcal{P}}$

4 **return** $(\widehat{\mathbb{P}^{\mathfrak{r}}}, \widehat{V}, \widehat{\mathbf{V}}, \widehat{U})$

In the backward implementation, tuple operations are componentwise. For any tuple-valued child expression $F(\bullet)$, write

$$\mathrm{Avg}_{\mathsf{s}}[F(\bullet)] := \frac{1}{N(\mathsf{s})} \sum_{\mathsf{a} \in \mathfrak{E}_{\mathsf{s}}} F(\mathsf{sa})$$

and

$$\mathrm{Avg}_{\mathsf{s},\neg\mathsf{a}}[F(\bullet)] := \begin{cases} \dfrac{1}{N(\mathsf{s}) - 1} \displaystyle\sum_{\mathsf{b} \in \mathfrak{E}_{\mathsf{s}} \setminus \{\mathsf{a}\}} F(\mathsf{sb}) & \text{if } N(\mathsf{s}) > 1, \\ \bot & \text{if } N(\mathsf{s}) = 1. \end{cases}$$

For tuples z, h of the same shape, set

$$\mathrm{Mix}_\lambda(z, h) := \begin{cases} \lambda z + (1 - \lambda) h & \text{if } z \text{ is present}, \\ h & \text{if } z = \bot. \end{cases}$$

The notation $-_p$ denotes the full private-state vector indexed by $x_p \in \mathcal{X}_p$.

Algorithm 8: $\mathcal{O} \leftarrow \texttt{BranchedValue}(\mathfrak{S}, V, \mathbf{V}, U, r)$

`/* Backward Propagation of Value Estimators. */`

Parameters: $\mathfrak{S}^{\beta,\pi}$, sampled labeled tree with child counts N; V, $\mathbf{V}$, U, value heads; r, reach probabilities

Hyperparameters: λ, GAE horizon

Output: target table $\mathcal{O}$

Initialize target table $\mathcal{O} \leftarrow \emptyset$

for *terminal* $\mathsf{t} \in \mathfrak{S}^{\beta,\pi}$ *labeled by pre-terminal* $\tau^{\mathsf{pre-c}}$ **do**

Monte Carlo estimation of

$$\begin{aligned} \widehat{\mathbb{P}}(x_p \mid \mathsf{t}) &\simeq \mathbb{P}^{\pi}(x_p \mid \tau^{\mathsf{pre-c}}) \\ \widehat{V}_p(x_p, \mathsf{t}) &\simeq \mathbb{E}^{\pi}(R_p(x,\tau) \mid \tau^{\mathsf{pre-c}}, x_p) \\ \widehat{U}(\mathsf{t}) &\simeq \mathbb{E}^{\pi}(\|\mathbf{R}(x,\tau)\|_2^2 \mid \tau^{\mathsf{pre-c}}) \\ \widehat{\mathbf{V}}(\mathsf{t}) &\simeq \mathbb{E}^{\pi}(\mathbf{R}(x,\tau) \mid \tau^{\mathsf{pre-c}}) \end{aligned} \tag{3.27}$$

for $p \in \mathcal{P}$ and $x_p \in \mathcal{X}_p$ using $\texttt{PreTerminalMomentMC}(\mathfrak{r})$

end

for *node* $\mathsf{s} \in \mathfrak{S}^{\beta,\pi}$ *non-terminal, backward* **do**

$\sigma \leftarrow \ell(\mathsf{s})$

Compute

$$\left(\left(\widehat{\mathbb{P}}(-_p \mid \mathsf{s}), \widehat{V}_p(-_p, \mathsf{s})\right)_{p\in\mathcal{P}}, \widehat{\mathbf{V}}(\mathsf{s}), \widehat{U}(\mathsf{s})\right) \leftarrow \mathrm{Avg}_{\mathsf{s}}\left[\left(\left(\widehat{\mathbb{P}}(-_p \mid \bullet), \widehat{V}_p(-_p, \bullet)\right)_{p\in\mathcal{P}}, \widehat{\mathbf{V}}(\bullet), \widehat{U}(\bullet)\right)\right] \tag{3.28}$$

if $o(\sigma) \in \mathcal{P}$ **then**

$\left(\widehat{Q}^{\lambda}(-_{o(\sigma)}, \mathsf{sa})\right)_{\mathsf{a}\in\mathfrak{E}_{\mathsf{s}}} \leftarrow \left(\widehat{V}_{o(\sigma)}(-_{o(\sigma)}, \mathsf{sa})\right)_{\mathsf{a}\in\mathfrak{E}_{\mathsf{s}}}$

$\widehat{V}_{o(\sigma)}(-_{o(\sigma)}, \mathsf{s}) \leftarrow \mathrm{Avg}_{\mathsf{s}}\left[\widehat{Q}^{\lambda}(-_{o(\sigma)}, \bullet)\right]$

$\left(\widehat{V}^{\lambda,\neg\mathsf{a}}(-_{o(\sigma)}, \mathsf{s})\right)_{\mathsf{a}\in\mathfrak{E}_{\mathsf{s}}} \leftarrow \left(\mathrm{Mix}_{\lambda}\left(\mathrm{Avg}_{\mathsf{s},\neg\mathsf{a}}\left[\widehat{Q}^{\lambda}(-_{o(\sigma)}, \bullet)\right], V(-_{o(\sigma)}, \sigma)\right)\right)_{\mathsf{a}\in\mathfrak{E}_{\mathsf{s}}}$

$\left(\widehat{A}^{\lambda}(-_{o(\sigma)}, \mathsf{sa})\right)_{\mathsf{a}\in\mathfrak{E}_{\mathsf{s}}} \leftarrow \left(\widehat{Q}^{\lambda}(-_{o(\sigma)}, \mathsf{sa}) - \widehat{V}^{\lambda,\neg\mathsf{a}}(-_{o(\sigma)}, \mathsf{s})\right)_{\mathsf{a}\in\mathfrak{E}_{\mathsf{s}}}$

Store current targets $\mathcal{O} \leftarrow \mathcal{O} \sqcup \left(\widehat{\mathbb{P}}(-_{o(\sigma)} \mid \mathsf{s}), (\widehat{A}^{\lambda}(-_{o(\sigma)}, \mathsf{sa}), \widehat{Q}^{\lambda}(-_{o(\sigma)}, \mathsf{sa}))_{\mathsf{a}\in\mathfrak{E}_{\mathsf{s}}}, \widehat{V}_{o(\sigma)}(-_{o(\sigma)}, \mathsf{s}), \widehat{\mathbf{V}}(\mathsf{s}), \widehat{U}(\mathsf{s})\right)$

Overwrite with the bootstrapped values propagated to ancestors:

$$\left(\widehat{V}_{o(\sigma)}(-_{o(\sigma)}, \mathsf{s}), \widehat{\mathbf{V}}(\mathsf{s}), \widehat{U}(\mathsf{s})\right) \leftarrow \mathrm{Mix}_{\lambda}\left(\left(\widehat{V}_{o(\sigma)}(-_{o(\sigma)}, \mathsf{s}), \widehat{\mathbf{V}}(\mathsf{s}), \widehat{U}(\mathsf{s})\right), \left(V(-_{o(\sigma)}, \sigma), \mathbf{V}(\sigma), U(\sigma)\right)\right) \tag{3.29}$$

end

end

return *target table* $\mathcal{O}$

Note that returning $\widehat{Q}$ is only necessary if we are using a `NeuRD` update rule. Otherwise $\widehat{A}$ suffices.

Algorithm 9: $(\theta_k)_{k\geq 0} \leftarrow$ `BranchedRaoBlackwellPG`(θ_0)

/* Tree-valued generalization of the sequential Rao-Blackwellized template; $\beta \equiv 1$ recovers it exactly. */

Parameters: θ_0, initial weights; θ, working copy of the next iterate

Hyperparameters: N, tree count; B, minibatch size; M, node budget; λ; $\beta_{\max}$; δ, support target

Hyperparameters: $\beta_{\texttt{fix}}$, $\beta_{\texttt{avg}}$, $k_{\texttt{warm}}$; chosen number of gradient-descent steps per iterate

Output: $(\theta_k)_{k\geq 0}$

for $k = 0, 1, 2, \ldots$ **do**

  Define $\beta_k = \beta_k^{U,\bar{V}}$ from frozen public moment heads $U^{\theta_k}, \mathbf{V}^{\theta_k}$ and previous-bucket means.

  Sample $((\mathfrak{S}, \ell : \mathfrak{S} \to \mathcal{S}, N : \mathfrak{S} \to \mathbb{N})$ using `SampleTree`$(\pi^{\theta_k}, \beta_k)$

  Compute target table $\mathcal{O} \leftarrow$ `BranchedValue`$(\mathfrak{S}, V^{\theta_k}, \mathbf{V}^{\theta_k}, U^{\theta_k}, r)$

  for *each gradient-descent step* **do**

    Sample $(P_b, \mathsf{S}_b)_{b=1}^B \overset{\text{iid}}{\sim} (P, \mathsf{S})$ and set $\Sigma_b = \ell(\mathsf{S}_b)$

    Deduce $\widehat{A}_b^\lambda(x_{P_b}, \mathsf{a})$, $\widehat{Q}_b^\lambda(x_{P_b}, \mathsf{a})$, $\widehat{V}_b(x_{P_b})$, $\widehat{\mathbf{V}}_b$ and $\widehat{U}_b$ from $\mathcal{O}$ at S_b

    Estimate

$$\begin{aligned}\widehat{l}(\theta) = \frac{1}{B}\sum_{b=1}^{B}\sum_{\substack{x_{P_b}\\ \mathsf{a}\in\mathfrak{C}_{\mathsf{S}_b}}} \frac{1}{N(\mathsf{S}_b)}\mathcal{L}^{\pi}\Big((\pi^\theta, V^\theta, \pi^{\theta_k}, V^{\theta_k})(x_{P_b}, \Sigma_b), a_{\mathsf{a}}, \widehat{A}_b^\lambda(x_{P_b}, \mathsf{a}), \widehat{Q}_b^\lambda(x_{P_b}, \mathsf{a})\Big)\widehat{\mathbb{P}}(x_{P_b} \mid \mathsf{S}_b)\\ + \frac{1}{B}\sum_{b=1}^{B}\sum_{x_{P_b}} \mathcal{L}^{V}\Big((V^\theta, V^{\theta_k})(x_{P_b}, \Sigma_b), \widehat{V}_b(x_{P_b})\Big)\widehat{\mathbb{P}}(x_{P_b} \mid \mathsf{S}_b)\\ + \frac{1}{B}\sum_{b=1}^{B} \mathcal{L}^{\mathbf{V},U}\Big((\mathbf{V}^\theta, U^\theta, \mathbf{V}^{\theta_k}, U^{\theta_k})(\Sigma_b), \widehat{\mathbf{V}}_b, \widehat{U}_b\Big)\end{aligned} \tag{3.30}$$

    Descend θ along $\nabla_\theta \widehat{l}(\theta)$

  end

  Store $\theta_{k+1} \leftarrow \theta$

end

return $(\theta_k)_{k\geq 0}$

Variance-Minimizing Sampling. The goal of this paragraph is to formalize and solve the problem of minimizing sampling variance. Typically, we want to pick the β intensities so to:

1. **Minimize:** the average variance $\mathbb{E}(\mathrm{Var}(\widehat{V}^1 \mid \Sigma))$ of the $\lambda = 1$ Monte Carlo value estimator,
2. **Subject to constraint:** $\mathbb{E}(|\mathfrak{S}|) = \mathbb{E}(\prod_{\mathsf{s}\preccurlyeq\mathsf{t}} N(\mathsf{s})) \leq M$ for some budget M.

This is obviously not the true variance of the loss estimator, but it is probably the best proxy we can get for it. To accomplish this, we can either:

1. Learn β^θ directly using some sort of loss function,
2. Learn other heads (such as U and $\mathbf{V}$ above) and use them to compute the chosen β intensities.

4 Poker: a Perfect Example

We focus on 6-max Texas Hold'em cash poker, no ante, straddle or rake, with exactly 100 big blinds per player. We don't simplify the game any further: all actions are allowed. Below are a few core variables:

$$
\begin{aligned}
|\mathcal{P}| &= 6 \\
\mathcal{P} &= \{\texttt{sb}, \texttt{bb}, \texttt{utg}, \texttt{hij}, \texttt{lo}, \texttt{btn}\} \\
\mathcal{D} &= \{0, \dots, 51\} \\
\mathcal{X}_p = \mathcal{H} &= \binom{\mathcal{D}}{2} \\
\mu &= \mathsf{Unif}(\{x \in \mathcal{X} \mid \sqcup_p x_p\}) \\
\mathsf{B} &\in \binom{\mathcal{D}}{3} \text{ or } \binom{\mathcal{D}}{4} \text{ or } \binom{\mathcal{D}}{5} \\
\mathsf{C} &= \sqcup_p x_p \sqcup \mathsf{B} \\
\chi(\cdot \mid x, \text{flop}) &= \mathsf{Unif}\left(\binom{\mathcal{D} \setminus \mathsf{C}}{3}\right) \\
\chi(\cdot \mid x, \text{turn}) &= \mathsf{Unif}\left(\binom{\mathcal{D} \setminus \mathsf{C}}{1}\right) \\
\chi(\cdot \mid x, \text{river}) &= \mathsf{Unif}\left(\binom{\mathcal{D} \setminus \mathsf{C}}{1}\right)
\end{aligned}
$$

Several specificities impact our implementation:

1. We feed the model **redundant** information by feeding it both the public state σ and the Bayesian-updated ranges $\mathfrak{r}$.

2. The game of poker has a strong **preflop gauge symmetry**: any suit permutation should leave the strategy invariant. Instead of 1326 possible private player states, there are 169. We exploit this by having a separate model for preflop and postflop, making it impossible for the model to not pick actions invariants under this transformation preflop. Optionally, we can also exploit this symmetry in the Monte Carlo estimators (see below) by storing the 4! suit permutations of each sampled private state and board into the accumulated statistics (this can be considered if possible with minimal wrap divergence).

3. The **preflop gauge symmetry** is harder to handle as it acts on the board which is an input. Optionally, we can add a loss function term that: samples a random suit permutation, applies it to the model inputs, and penalizes the difference between the re-permutated outputs and the original outputs. This adds one forward pass per sample, which shouldn't be too expensive compared to the cost of a backward pass.

4. In **postflop** states, to control the size of the input, we actually only feed the ranges of the **live** players (meaning those who have not folded yet and are thus still eligible to win part of the pot). We feed the model information about folded players by exploiting the very architecture of attention heads (see subsection to come on cluster expansions and Möbius expansions). This adds a computational step which can be seen as the first step of the model (and can be launched in the same batches as the model itself).

5. The model is a multi-headed attention transformer model, with one token per pair (p, h) (for p only a live player postlfop), one token for each piece of public information, and one taken per **action mode**. The tokens corresponding to action a and pair $(p_{\text{acting},h})$ are used to output propensity for a to be played with hand h.

6. As a consequence, we need to either:
   (a) **Memory Pressure:** during rollout, store the live ranges of each rolled-out state, so they can be fed to the minibatch samples as inputs
   (b) **Redundant Computations:** during minibatches, recompute the live ranges of each samples state Σ by quickly walking back the tree from Σ to $\emptyset$, and doing $\odot$-Bayesian updates along the way.
7. Three operations are already implemented in `CUDA` and `C++` in a separate repository:
   (a) The **folded statistics** to be fused with the model as its first step. This is **only** used: right after a player folded (otherwise can be stored and passed down the trajectory), and postlfop (preflop we can afford feeding the model all ranges).
   (b) A **rejection sampler** to be used exclusively for the branching rollout, and to be fused with the model as its last step,
   (c) A **Monte Carlo estimator** of equity and posteriors, which will be used for both quantum versions after rollouts are completed, and before backpropagation to compute $\widehat{Q}$, $\widehat{V}$, ...

   Folded statistics might be more efficiently computed by the CPU considering that they are only randomly necessary. The other two operations should really be GPU-based.
8. We implement a synchronous rollout that is heavily computed by GPUs (typically a node of 2-4). We maintain a queue of states to be rolled out, and we roll them out in batches (two types of batches: one for preflop, one for postflop).
9. **Loss Function:** we a clipped `PPO` loss, with an optional `KL` term to an older reference index $\mathrm{ref}(k)$, an entropy term to encourage exploration (to be faded away throughout training), and loss terms for each of the value heads:

$$\mathcal{L}_{\texttt{clip}}[\widehat{A}^{\lambda}] + \mathcal{L}_{\texttt{H}} + \mathcal{L}_{\texttt{KL}} + \mathcal{L}_{\texttt{V}}[\widehat{V}^{\lambda}] + \mathcal{L}_{\texttt{U}}[\widehat{U}^{\lambda}] + \mathcal{L}_{\mathbf{V}}[\widehat{\mathbf{V}}^{\lambda}]$$

   Ideally, the implementation should allow the user to pick any loss function.
10. Ideally, the implemtation should be clearly modular: it will be used for poker, but must be easily adaptable to other PBGs.